

PokeBuilder - Aplicación para gestión de equipos competitivos Pokémon



Héctor Martínez, Daniel Peñalver, Joel Gallo y Álvaro López

Desarrollo de Aplicaciones Multiplataforma

Memoria del Proyecto de DAM

IES Abastos. Curso 2025/2026.

Grupo 2ºDAM.

20 de Febrero de 2026

Índice

1. Identificación, justificación y objetivos	3
1.1 Identificación	3
1.2 Justificación	3
1.3 Objetivos	4
2. Marcos tecnológicos y fundamentos teóricos	5
2.1 Arquitectura Cliente-Servidor	5
2.2 Arquitectura REST y formato JSON	5
2.3 Autenticación basada en JWT	6
2.4 Integración con PokeAPI	6
3. Diseño del proyecto	7
3.1 Arquitectura general	7
3.2 Diseño de base de datos	7
3.3 Diseño de seguridad	8
3.4 Diseño de la aplicación Android	8
3.5 Diseño de la aplicación Electron	9
3.6 Colores	9
4. DESARROLLO DEL PROYECTO	10
4.1 Backend	10
4.2 Integración con PokeAPI	10
4.3 Gestión competitiva	11
4.4 Gestión de equipos	11
4.5 Selección de movimientos	12
5. PRUEBAS Y VALIDACIÓN	12
6. EVALUACIÓN Y CONCLUSIONES	13
6.1 Resultados	13
6.2 Dificultades	13
6.3 Aprendizaje	14
7. REFERENCIAS	14
Anexo de imágenes	15
Versión móvil	15
Versión de escritorio	16

1. *Identificación, justificación y objetivos*

1.1 Identificación

El presente proyecto consiste en el desarrollo de un sistema completo basado en una arquitectura cliente-servidor orientado a la gestión de equipos competitivos de la saga de videojuegos Pokémon. La solución está compuesta por un backend REST desarrollado con Spring Boot, una aplicación móvil Android implementada en Kotlin utilizando Jetpack Compose, una aplicación de escritorio con Electron, una base de datos relacional para la persistencia de la información y la integración con PokeAPI como una fuente externa de datos oficiales.

El sistema permite que los usuarios autenticados creen, modifiquen y almacenen equipos Pokémon personalizados. Cada criatura puede configurarse con parámetros avanzados propios del entorno competitivo, como habilidades, movimientos, objetos equipados, valores individuales (IV), valores de esfuerzo (EV) y naturaleza. Además, el sistema calcula automáticamente las estadísticas finales aplicando las fórmulas oficiales del videojuego, garantizando coherencia matemática y competitiva.

El proyecto no se limita a una simple aplicación informativa, sino que implementa una solución estructurada, segura y escalable, alineada con prácticas profesionales de desarrollo de software.

1.2 Justificación

El entorno competitivo de Pokémon exige una planificación avanzada para que sus participantes tengan un éxito mayor. Las estadísticas finales de cada criatura no dependen únicamente de sus estadísticas base, sino que de un conjunto de variables matemáticas que se relacionan entre sí: los valores individuales (IV), los valores de esfuerzo (EV) y las naturalezas. La correcta optimización de estos parámetros puede determinar el rendimiento de un equipo dentro de una batalla.



Anexo A, batalla competitiva de Pokemon Scarlet/Violet

Este proyecto surge como una solución integral que permite:

- Control total sobre la arquitectura del sistema.
- Implementación de seguridad avanzada mediante autenticación JWT
- Persistencia estructurada por usuario.
- Extensibilidad futura.
- Aplicación práctica de conocimientos adquiridos durante el ciclo formativo.

Desde el punto de vista académico, el proyecto permite integrar múltiples competencias del ciclo de Desarrollo de Aplicaciones Multiplataforma: diseño de APIs REST, gestión de bases de datos, programación orientada a objetos, consumo de servicios externos y desarrollo de aplicaciones de escritorio y móviles.

1.3 Objetivos

El objetivo general del proyecto es desarrollar una plataforma conectada a un backend funcional y seguro que permita la creación, configuración y almacenamiento de una gran variedad de equipos Pokémon competitivos.

Para alcanzar este mismo objetivo general se establecieron los siguientes objetivos específicos:

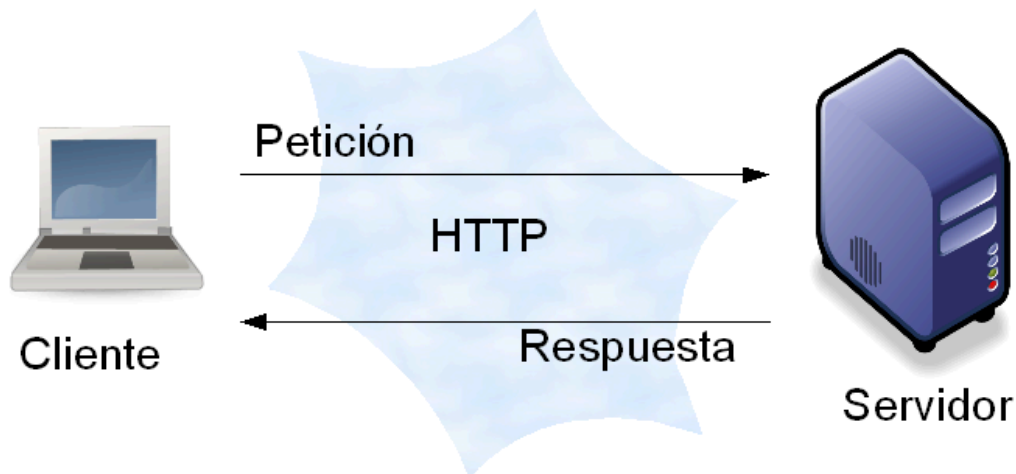
- Implementar un sistema de autenticación basado en JSON Web Token.
- Diseñar una base de datos relacional normalizada.
- Aplicar una arquitectura en capas en el backend.
- Consumir servicios externos REST (PokeAPI).
- Implementar validaciones conforme a reglas competitivas oficiales.
- Diseñar una interfaz móvil moderna y reactiva basada en Jetpack Compose.
- Desarrollar software de escritorio con Electron.

Al haber alcanzado estos objetivos habremos llegado al objetivo final de tener las aplicaciones deseadas que queremos para este proyecto

2. Marcos tecnológicos y fundamentos teóricos

2.1 Arquitectura Cliente-Servidor

El sistema adopta un modelo cliente-servidor claramente diferenciado. Las aplicaciones de Android y Electron actúan como cliente y consumen servicios REST expuestos por el backend desarrollado en Spring Boot y alojado en Render. La comunicación se realiza mediante protocolo HTTP utilizando formato JSON para la serialización de datos.



Anexo B, ejemplo simple de la arquitectura Cliente-Servidor

Este modelo favorece la separación de responsabilidades, permitiendo desacoplar completamente la lógica de negocio del cliente. Además, facilita la escalabilidad del sistema y la posibilidad de integrar futuros clientes adicionales sin modificar la lógica del servidor.

2.2 Arquitectura REST y formato JSON

La API del backend sigue principios RESTful. Cada recurso del sistema (como usuarios, equipos, pokémon, etc..) está identificado mediante URLs claras y estructuradas, y se manipula mediante métodos HTTP estándar (GET, POST, PUT, DELETE).

Un ejemplo podría ser el siguiente:

- GET/api/pokemon
- GET/api/pokemon/id/3
- DELETE/api/pokemon
- DELETE/api/pokemon/id/2

La representación propia de datos se realiza en el formato JSON, lo que garantiza interoperabilidad, legibilidad y compatibilidad con múltiples plataformas.

2.3 Autenticación basada en JWT

El sistema implementa autenticación mediante JSON Web Token. Ya que tras un proceso de “login” correcto, el servidor genera un token firmado digitalmente que contiene la identidad del usuario. Dicho token debe enviarse en la cabecera Authorization en cada petición protegida.

La validación del token se realiza mediante un filtro personalizado donde intercepta las peticiones antes de su procesamiento por parte de Spring Security.

Este enfoque presenta varias ventajas en frente de otras herramientas que se podrían haber usado en este proyecto:

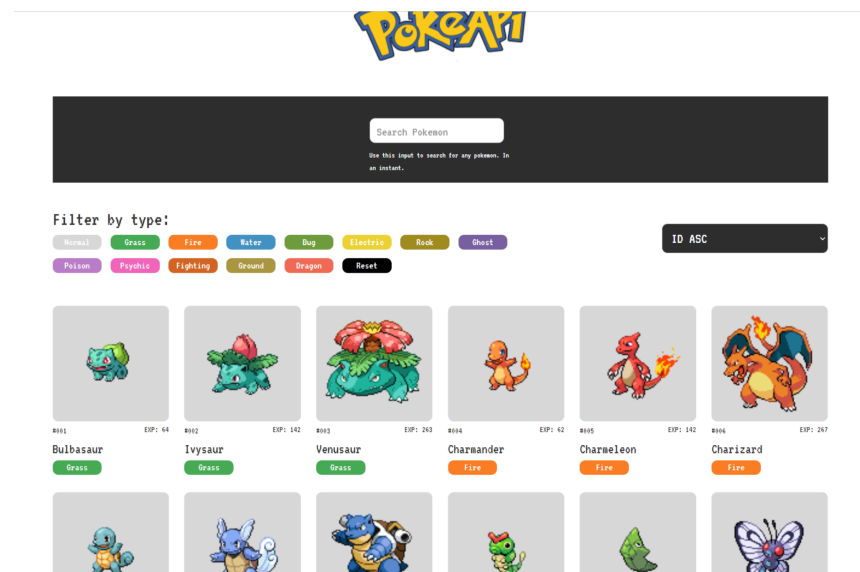
- Arquitectura stateless (sin sesiones en servidor).
- Mayor escalabilidad.
- Separación clara entre autenticación y lógica de negocio.
- Reducción de consumo de recursos en servidor.

2.4 Integración con PokeAPI

PokeAPI es una API pública que proporciona datos oficiales del universo Pokémon, incluyendo las estadísticas base, los movimientos, las habilidades, sus tipos y las imágenes que se les mostrará al usuario.

En lugar de almacenar toda esta información en la base de datos local, el sistema consulta dinámicamente la API externa, reduciendo la redundancia y manteniendo consistencia con los datos oficiales.

La integración se realiza mediante el consumo HTTP y el procesamiento de estructuras JSON complejas.



Anexo C, ejemplo de uso de la PokeAPI dentro de otro proyecto

3. Diseño del proyecto

3.1 Arquitectura general

El backend se organiza siguiendo una arquitectura de 4 capas diferenciadas entre sí:

- Controller: expone los endpoints REST.
- Service: contiene la lógica de negocio.
- Repository: gestiona el acceso a datos mediante JPA.
- Model y DTO: representan entidades persistentes y objetos de transferencia.

En la aplicación Android se emplea la estructura Clean Architetcure basada en separación de responsabilidades:

- Capa de UI implementada con Compose.
- Repository para gestión de datos.
- ApiService con Retrofit para consumo REST.
- AuthManager para gestión de autenticación.
- Interceptor JWT mediante OkHttp.

En la aplicación de Electron el proceso principal gestiona el ciclo de vida de la ventana, aplica la seguridad de contenidos y expone un sistema de almacenamiento persistente en JSON para el token JWT y los datos de sesión mientras que el proceso de renderizado se organiza en vistas HTML independientes, cada una con su propio módulo JavaScript.

Esta organización favorece el correcto mantenimiento, testeo y escalabilidad del proyecto.

3.2 Diseño de base de datos

Se diseña una base de datos relacional normalizada basada en PostgreSQL y alojada en Render, compuesta por tres entidades principales: User, Team y Pokémon.

Un usuario puede tener múltiples equipos, y cada equipo puede contener múltiples Pokémon, estableciendo una relación OneToMany entre User y Team y ManyToOne entre Pokémon y Team.

Las contraseñas se almacenan cifradas mediante BCrypt, garantizando seguridad frente a filtraciones.

El modelo evita redundancia innecesaria, delegando la información oficial a PokeAPI y almacenando únicamente configuraciones personalizadas del usuario.

3.3 Diseño de seguridad

El backend se configura con Spring Security bajo la política de SessionCreationPolicySTATELESS, lo que implica que no se mantienen sesiones activas en el servidor.

Se implementa un filtro JWT personalizado que se ejecuta antes del UsernamePasswordAuthenticationFilter. Las rutas públicas incluyen autenticación, comprobación de estado del sistema y documentación con Swagger, mientras que el resto de endpoints requieren token válido.

Esta configuración garantiza protección frente a accesos no autorizados, aislamiento por usuario y gestión correcta de errores HTTP.



Anexo B, el diseño final de la seguridad e implementación dentro de la aplicación de escritorio.

3.4 Diseño de la aplicación Android

La interfaz se desarrolla utilizando Jetpack Compose, un kit de herramientas de Google que permite gestionar el estado de forma reactiva y reducir complejidad respecto al sistema XML tradicional.

Para el consumo de la API se emplea Retrofit junto con el sistema de corrutinas de Kotlin para la correcta gestión de las funciones asíncronas. El token JWT se inyecta automáticamente en cada petición mediante un interceptor de OkHttp.

La arquitectura elegida mejora la claridad del código, reduce acoplamientos y facilita la evolución futura.

3.5 Diseño de la aplicación Electron

La interfaz se desarrolla con HTML5, CSS3 y JavaScript vanilla, complementados con Bootstrap 5 como sistema de componentes y rejilla.

La gestión de autenticación actúa como puente seguro entre ambos procesos, exponiendo al frontend métodos para manejar el token JWT sin conceder acceso directo a las APIs de Node.js. Dicho token se inyecta automáticamente en cada petición para todos los módulos. La protección de rutas se implementa en cada vista, redirigiendo al usuario al login si no existe sesión activa.

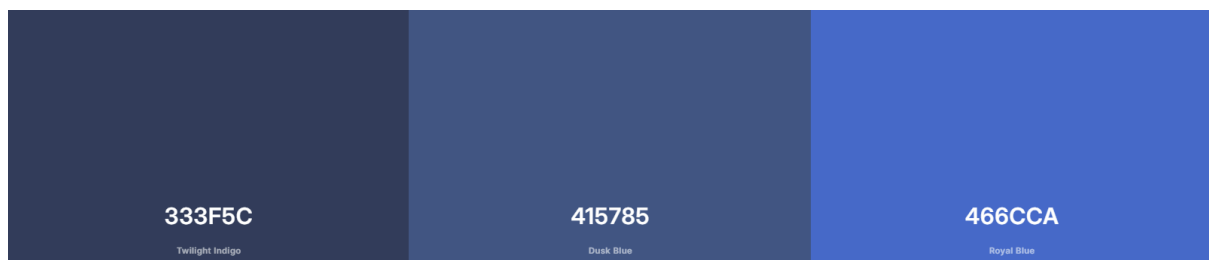
El consumo de datos externos se realiza mediante Fetch API contra el backend propio desplegado en Render. La arquitectura elegida separa las responsabilidades de cada vista y facilita la incorporación de nuevas funcionalidades sin alterar el núcleo de la aplicación.

3.6 Colores

Para tanto la aplicación de Electron como la de Android se optó por escoger una específica paleta de colores, siendo esta varios tonos de azules, cambiando el fondo de la aplicación cuando se selecciona un pokémon dependiendo de sus tipos, con un solo color uniforme si solo tiene 1 tipo y si tiene 2 tipos se observará un gradiente entre los dos colores que representan los tipos.

Un ejemplo de lo mencionado es si se escoge a Venusaur, un pokémon tipo planta/veneno, como resultado se verá que la pantalla ahora tendrá un gradiente entre verde (el color que representa el tipo planta) y el morado (que representa el tipo veneno).

Se seleccionó esta paleta por su vista agradable y buena combinación con otros colores vistos en la aplicación.



Anexo B, paleta de colores de <https://coolers.co/333f5c-415785-466cca>

4. DESARROLLO DEL PROYECTO

4.1 Backend

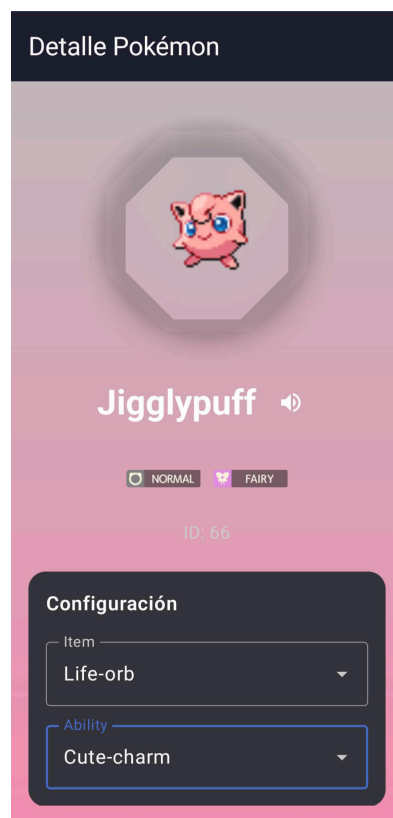
El backend se implementa en render con Spring Boot 3, JPA/Hibernate y MySQL como sistema de persistencia. Se desarrollan DTOs para desacoplar las entidades internas de las respuestas enviadas al cliente, evitando exponer directamente el modelo de base de datos.

4.2 Integración con PokeAPI

Se desarrolla un servicio específico encargado de:

- Obtener datos generales del Pokémon.
- Extraer imagen oficial.
- Determinar tipos primario y secundario.
- Procesar estadísticas base.

El procesamiento del JSON requiere manejo dinámico de estructuras anidadas mediante Map<String, Object>.



Anexo E, Se seleccionó un pokemon y ahora se muestran sus detalles y configuración, como sus tipos

4.3 Gestión competitiva

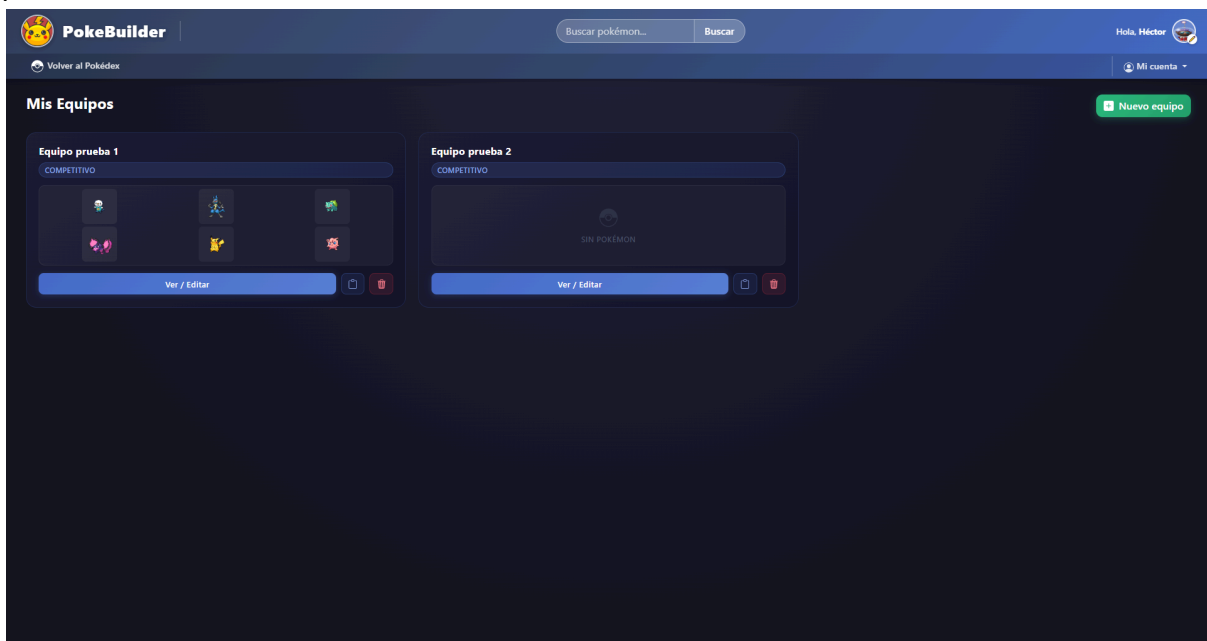
Se implementan validaciones conforme a normativa oficial:

- EV máximo por estadística: 252.
- EV total máximo: 510.
- IV en rango 0–31.

Las validaciones se aplican tanto en frontend como en backend para garantizar coherencia y evitar manipulaciones indebidas.

4. Gestión de equipos

En la pantalla de equipos se puede agregar tantos equipos como se desee y añadir hasta un máximo de seis pokémon por equipo. Cada tarjeta cuenta con un botón para exportar la información del equipo en formato texto al portapapeles, un botón para borrar y otro para editar los pokémon que componen cada equipo. Se escogió esta forma de exportar ya que páginas famosas como Pokémon Showdown (página web para competir online) utilizan un formato de texto concreto para importar equipos. Además se integró en la aplicación la posibilidad de subir imágenes, clicando arriba a la derecha se puede cambiar la foto de perfil del usuario.



4.5 Selección de movimientos

Se desarrolla un endpoint específico para obtener movimientos disponibles según el Pokémon seleccionado. Los resultados se procesan eliminando duplicados y ordenándolos alfabéticamente.

En Android se implementa un diálogo con buscador para facilitar la selección eficiente.

Anexo F, Apartado de estadísticas del pokémon

Detalle Pokémon

STEEL DRAGON

ID: 76

Configuración

Item

Ability

Movimientos

Seleccionar movimiento

Seleccionar movimiento

Seleccionar movimiento

Seleccionar movimiento

Detalle Pokémon

IVs (0-31)

HP 31 Atk 31

Def 31 SpA 31

SpD 31 Spe 31

EVs (0-252)

Total EV: 0 / 510

HP 0 Atk 0

Def 0 SpA 0

SpD 0 Spe 0

5. PRUEBAS Y VALIDACIÓN

Para saber y comprobar el correcto funcionamiento de ambas aplicaciones se tuvieron que hacer algunos tipos de pruebas para saber si funciona correctamente, destacando las siguientes pruebas:

- Verificación de autenticación correcta e incorrecta, un requerimiento crucial ya que el mal funcionamiento de una función de seguridad puede resultar en el acceso no restringido de usuarios a sistemas no deseados.

- Intentos de acceso sin token, siguiendo con la prueba anterior, se tiene que probar esta función dada su papel vital en la seguridad y su uso en el restringir el acceso de usuarios no queridos a la aplicación..
- Configuraciones inválidas de EV para que haya un límite de la cantidad y distribución que los usuarios pueden poner a los Evs de un mismo pokémon.
- Cálculo correcto de estadísticas, ya que las estadísticas dependen del cálculo correcto de diferentes características, y si su funcionamiento resulta erróneo, puede ocasionar algunos errores graves.
- La persistencia posterior al reinicio del servidor, para así saber si la información se guardaba correctamente

Con las confirmaciones y los éxitos de cada una de estas pruebas se valida el mantenimiento funcional de la integridad relacional, la correcta gestión de las excepciones HTTP y el correcto funcionamiento de las propiedades de seguridad al no permitir acceso a usuarios no autorizados.

6. EVALUACIÓN Y CONCLUSIONES

6.1 Resultados

El proyecto culmina en una plataforma funcional compuesta por un backend seguro y estructurado, una aplicación móvil moderna e integración con API externa

La aplicación permite crear y gestionar equipos completos con una gran variedad de variables que pueden hacer fácilmente cualquier equipo único del uno al otro.

Teniendo que manejar de los propios pokémon: las habilidades, las naturalezas, sus propias características y los ítems que pueden llevar.

6.2 Dificultades

Aunque el proyecto fue completado a tiempo, se pudieron observar algunas dificultades técnicas que impidieron el temporalmente el desarrollo fluido del proyecto. Las más destacadas son las siguientes:

- La configuración avanzada de Spring Security y sus funciones relacionadas con la seguridad.
- La implementación correcta del flujo JWT en Android.
- La sincronización entre entidades y DTOs dentro de las diferentes aplicaciones de nuestro proyecto.

- La gestión de varios errores 403 en entornos desplegados, sobre todo a la hora de implementar las funciones de seguridad.
- La implementación conjunta de los módulos claves (como Acceso a Datos y Desarrollo de Interfaces), dentro del proyecto.

Aunque como ya se ha mencionado antes, se pudieron al final resolver sin ningún daño terminal al propio proyecto.

6.3 Aprendizaje

Tanto en el desarrollo del proyecto como en su finalización se consolidó algunos conocimientos clave que se obtuvieron en las clases que se dieron durante todo este curso, agregadas algunas que el equipo obtuvo al crear esta aplicación.

Se pueden resumir estos conocimientos clave en:

- Arquitectura REST profesional.
- Seguridad aplicada en entornos reales.
- Diseño estructurado de software.
 - Diseño de software (Kotlin)
 - Diseño de software (JavaScript [Electron])
- Desarrollo full-stack móvil.
- Desarrollo con Bootstrap designado para el estilo.

7. REFERENCIAS

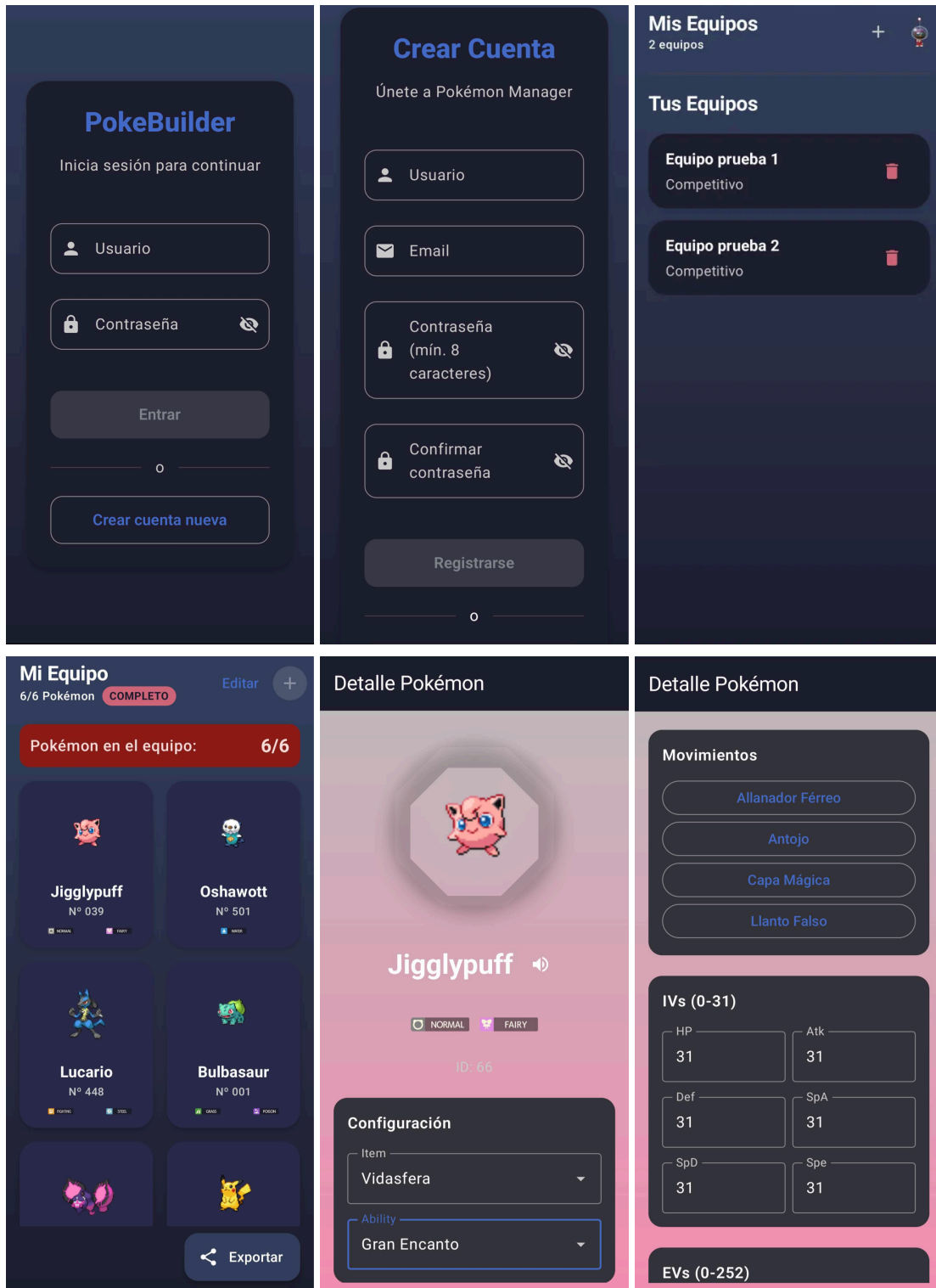
Finalmente, para la realización de este proyecto, se utilizaron las siguientes referencias como una ayuda esencial para la producción del mismo:

- [Documentación oficial de Spring Boot](#)
- [Documentación oficial de Android Developers](#)
- [PokeAPI](#)
- [OWASP JWT Security Guidelines](#)
- [Documentación oficial de Hibernate ORM](#)
- [Página web de Pokémon Showdown](#)

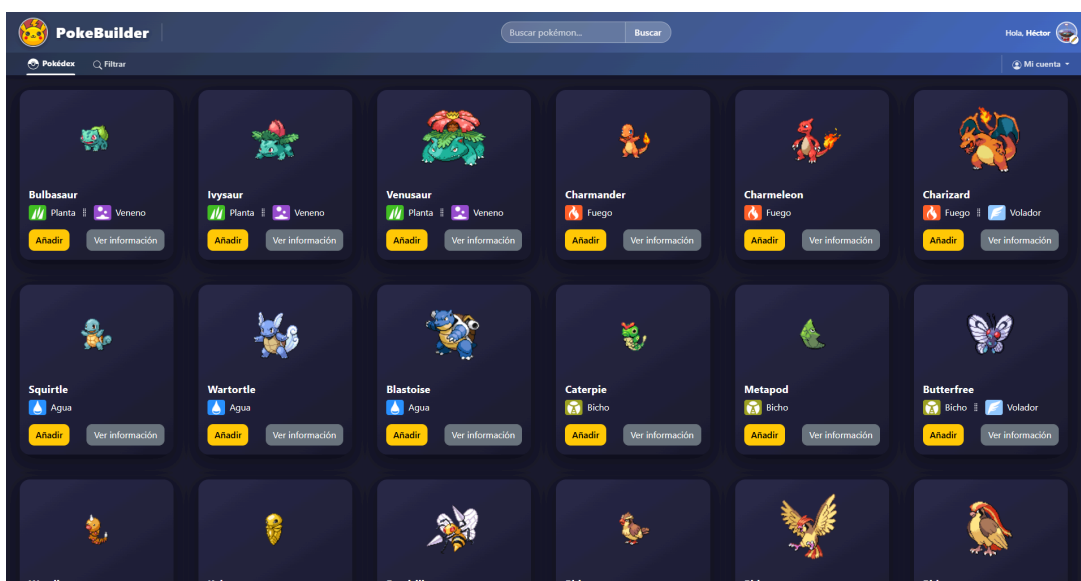
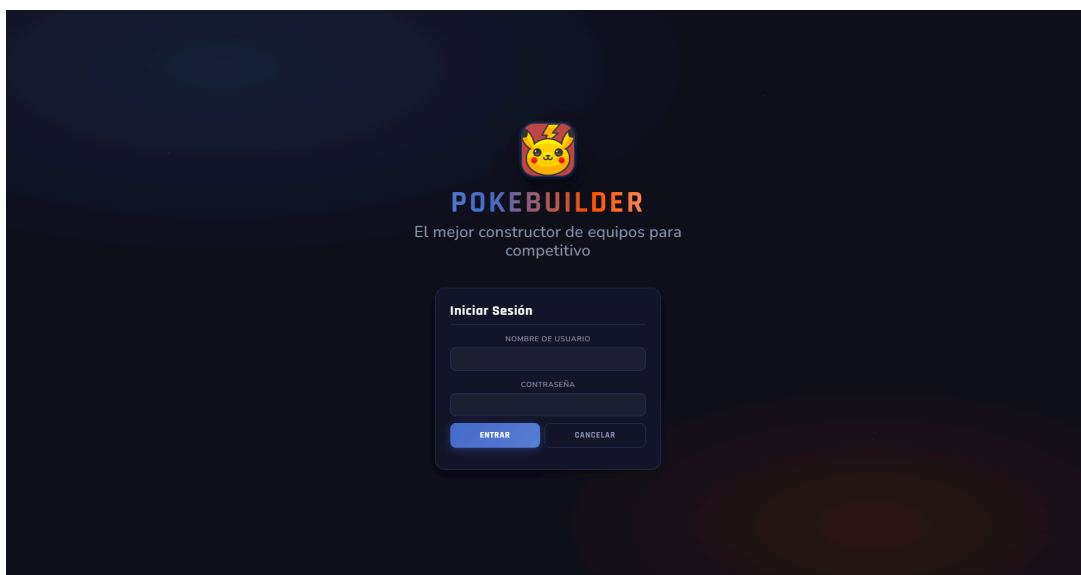
Anexo de imágenes

En este último apartado se aportan imágenes de todos los aspectos de la aplicación, tanto de la versión de móvil como de escritorio.

Versión móvil:



Versión de escritorio:



PokeBuilder

